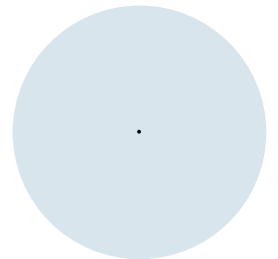


Fallunterscheidungen II



Bedingungsoperator

- Ternärer (dreistelliger) Bedingungsoperator bildet Ausdrucks-Alternative
- Syntax: `c ? a : b`
- Datentyp des Ausdrucks ist gemeinsamer Typ von `a` und `b`
- Wert des Ausdrucks ist
 - Wert von `a`, falls Wert von `c == true`
 - Wert von `b`, falls Wert von `c == false`
- Beispiel: Falls `j` kleiner als 10 ist, bekommt `i` den Wert 47, ansonsten bekommt `i` den Wert 11

```
int i = (j < 10) ? 47 : 11;
```



code06.

Condition
Operator2

Auswertungsreihenfolge

- Prioritäten für Operatoren, z.B. "Punkt vor Strich"
- Auswertungsreihenfolge für Operatoren gleicher Priorität ("Assoziativität")
 - rechts-assoziativ, Auswertung von rechts
 - links-assoziativ, Auswertung von links
- Per Klammerung () gewünschte Auswertungsreihenfolge immer bestimmbar
- Komponenten-zugriffe und Methodenaufrufe haben sogar noch höhere Priorität

Operator Precedence

Operators	Precedence
postfix	<i>expr</i> ++ <i>expr</i> --
unary	++ <i>expr</i> -- <i>expr</i> + <i>expr</i> - <i>expr</i> ~ !
multiplicative	* / %
additive	+ -
shift	<< >> >>>
relational	< > <= >= instanceof
equality	== !=
bitwise AND	&
bitwise exclusive OR	^
bitwise inclusive OR	
logical AND	&&
logical OR	
ternary	? :
assignment	= += -= *= /= %= &= ^= = <<= >>= >>>=

Geschachtelte if-Anweisungen

```
if (<Bedingung1>) {  
    <Anweisung>    // Falls Bedingung1 true  
} else if (<Bedingung2>) {  
    <Anweisung>    // Falls Bedingung1 false und  
                  // Bedingung 2 true  
} else if (<Bedingung3>) {  
    <Anweisung>    // Falls Bedingung1 false und  
                  // Bedingung2 false und  
                  // Bedingung3 true  
} else {  
    <Anweisung>    // Falls alle vorherigen  
                  // Bedingungen false  
}
```



code06.
IfElseElse2

switch-Anweisung

- wie `if`, aber mit mehr Alternativen
- kann u.U. geschachtelte `if`-Anweisungen ersetzen
- spart Schreibarbeit
- ist nicht unbedingt übersichtlicher als mehrere `if`-Anweisungen

```
switch (<Ausdruck>) {  
    case <Konstante> -> {  
        <Anweisungsfolge>  
    }  
    ...  
    case <Konstante> -> {  
        <Anweisungsfolge>  
    }  
    default -> {  
        <Anweisungsfolge>  
    }  
}
```

switch-Anweisung

- Ausdruck vom Typ **byte**, **short**, **int**, **long**, **char**, **String**
(oder eine Aufzählung/enum - kommt später)
- case-Block wird bei **Konstante, die dem Wert des Ausdrucks** entspricht ausgeführt
- Ohne passende Konstante wird der **default**-Block ausgeführt; default darf auch fehlen

```
switch (<Ausdruck>) {  
    case <Konstante> -> {  
        <Anweisungsfolge>  
    }  
    ...  
    case <Konstante> -> {  
        <Anweisungsfolge>  
    }  
    default -> {  
        <Anweisungsfolge>  
    }  
}
```



Switch1

switch-Anweisung, Besonderheiten

- Klammern bei **case** können weggelassen werden, wenn es nur eine Anweisung gibt



Switch2

```
switch (<Ausdruck>) {  
    case <Konstante> -> <Anweisung>  
    case <Konstante> -> <Anweisung>  
    :  
    case <Konstante> -> <Anweisung>  
    default -> <Anweisung>  
}
```

- Mehrere Konstanten können angegeben werden



Switch3

```
switch (<Ausdruck>) {  
    case <Konstante1>, <Konstante2> -> {  
        <Anweisungen>  
    }  
    :  
}
```

switch-Anweisung, Blöcke

- Wiederholung: { }-Paare definieren eigene Blöcke
- Auf Variablen eines Blockes kann nur im Block selbst oder in inneren Blöcken zugegriffen werden

```
switch (<Ausdruck>) {  
    case <Konstante> -> {  
        int b = 2;  
        <Anweisungsfolge>  
    }  
    case <Konstante> -> {  
        b = 3;  
        <Anweisungsfolge>  
    }  
}
```

ein Block

ein anderer Block

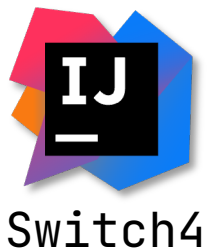
Compilerfehler!

switch-Anweisung mit Rückgabe

- switch-Ausdrücke können Werte zurückgeben
 - entweder direkt ohne Block
 - oder in Blöcken mittels des Schlüsselworts **yield**
- Praktisch für eine Fallunterscheidung, die je nach Wert eine Variable unterschiedlich setzt.

```
<Variable> = switch (<Ausdruck>) {  
    case <Konstante> -> <Rückgabe>  
    case <Konstante> -> {  
        <Anweisungsfolge>  
        yield <Rückgabe>  
    }  
    default -> <Rückgabe>  
};
```

Semikolon nicht vergessen!



switch vor Java 14

- links: Alte Version vor Java 14
- Wird immer noch unterstützt, ist aber fehleranfällig (wenn man das break vergisst, werden auch die Anweisungen aus dem folgenden case ausgeführt)

```
switch (<Ausdruck>) {  
    case <Konstante>:  
        <Anweisungsfolge>  
        break;  
    ...  
    case <Konstante>:  
        <Anweisungsfolge>  
        break;  
    default:  
        <Anweisungsfolge>  
}
```



besser

```
switch (<Ausdruck>) {  
    case <Konstante> -> {  
        <Anweisungsfolge>  
    }  
    ...  
    case <Konstante> -> {  
        <Anweisungsfolge>  
    }  
    default -> {  
        <Anweisungsfolge>  
    }  
}
```

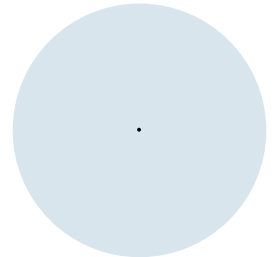
if und switch

- if ist die allgemeine Anweisung
 - Tests über logische Verknüpfungen, Zahlenbereiche, etc
- switch ist spezieller
 - testet nur auf Gleichheit mit einzelnen Werten
- **Jedes switch lässt sich in (verschachtelte) if-Anweisungen umwandeln**
- Umgekehrt ist das im Allgemeinen nicht möglich
 - und im Speziellen zwar manchmal möglich, aber aufwändig



code06.
IfIsMore
Generic

Final Variablen



Konstanten

- Variablen, deren Wert sich nicht verändern lassen könne soll werden **Konstanten** genannt.
 - Beispiele aus der Mathematik: π , e
- Konstanten werden in Java durch das Schlüsselwort **final** deklariert
- Konstanten, die bereits einen Wert besitzen, kann kein neuer Wert zugewiesen werden. In diesem Fall erzeugt der Compiler eine Fehlermeldung.
- Wenn möglich sollen **final** Variablen direkt bei der Deklaration initialisiert werden.
- **Final** Variablen werden groß geschrieben, z.B. PI, MAXIMALE_LAENGE.

```
final int BEISPIEL1 = 1;
final int BEISPIEL2;
BEISPIEL2 = 5;
// BEISPIEL1 = 6;

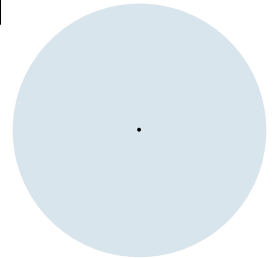
    System.out.println("BEISPIEL1:"
+BEISPIEL1);

    System.out.println("BEISPIEL2:"
+BEISPIEL2);
```



code06.
Konstanten

Stringmethoden



Zahlen, Buchstaben und boolean in String umwandeln

- `String.valueOf(<irgendein primitiver Wert>)` wandelt den Wert in einen String um

```
String zahlAlsString = String.valueOf(2.355); // "2.355"
```

- Passiert automatisch, wenn man Werte zu Strings hinzufügt

```
int zahl = 42;  
String text = "Meine Lieblingszahl: " + zahl;
```

kein `String.valueOf(zahl)` nötig

- "Trick": Wert zu leerem String hinzufügen, konvertiert diesen auch

```
String zahlAlsString = "" + 2.355; // "2.355"
```

- Aufpassen bei char: Ohne Umwandlung werden die Unicode-Werte addiert!

```
System.out.println('J' + 'S');           // Ausgabe 157  
System.out.println("" + 'J' + 'S');      // Ausgabe JS
```



code05.
ValueTo
String

Strings in Zahlen umwandeln

- `Integer.parseInt()`, `Byte.parseByte()`, `Long.parseLong()`, `Short.parseShort()`
- `Double.parseDouble()`, `Float.parseFloat()`
- auch `Boolean.parseBoolean()`
- Können Fehler (*Exception*) werfen, wenn das Format nicht stimmt
- Hinweis: `Double.parseDouble()` erwartet Double-Zahlen mit Punkt als Dezimaltrenner. Double-Zahlen mit Komma parsen geht über `NumberFormat`, siehe : <https://stackoverflow.com/a/4323628> (Nicht Teil der Prüfung!)



code05.
StringTo
Number

Auszug aus der String API

jshell>



StringMethods1

Return Type	Method	Description
int	length()	Returns the length of this string.
boolean	isEmpty()	Returns true if, and only if, length() is 0! Only true for the empty string "".
char	charAt(int index)	Returns the char value at the specified index.
String	substring(int beginIndex)	Returns a string that is a substring of this string, starting from beginIndex. Example: "abc".substring(1) is "bc".
String	substring(int beginIndex, int endIndex)	Returns a string that is a substring of this string. endIndex is not included. Example: "abc".substring(1,2) is "b".
boolean	equals(String s)	Compares this string to the specified String. Example: "abc".equals("abc") is true.

"GIRAFFEN"
01234567

Zeichen Nr. 3 ist A

Achtung: String der Länge 8,
aber Nummerierung beginnt bei 0

Strings nie mit == vergleichen,
sondern immer über .equals!!!



StringCompare

Auszug aus der String API

```
jshell>
```



code06.

StringMethods2

Return Type	Method	Description
<code>boolean</code>	<code>contains(String s)</code>	Returns true if and only if this string contains the specified String. Example: <code>("abb").contains("b")</code> and <code>("abb").contains("abb")</code> are true.
<code>boolean</code>	<code>startsWith(String prefix)</code>	Tests if this string starts with the specified prefix. Example: <code>("abb").startsWith("ab")</code> is true.
<code>boolean</code>	<code>endsWith(String suffix)</code>	Tests if this string ends with the specified suffix. Example: <code>("abb").endsWith("bb")</code> is true.
<code>int</code>	<code>indexOf(String str)</code>	Returns the index within this string of the first occurrence of the specified String. Example: <code>("abb").indexOf("b")</code> is 1. Returns -1 if the specified string is not part of this string.

"GIRAFFEN"
01234567

Auszug aus der String API

```
jshell>
```



StringMethods3

Return Type	Method	Description
String	toUpperCase()	Converts all of the characters in this String to upper case using the rules of the default locale. Example: ("ab!").toUpperCase() is "AB!".
String	toLowerCase()	Converts all of the characters in this String to lower case using the rules of the default locale. Example: ("AB!").toLowerCase() is "ab!".
String	strip() Es gibt auch trim(), man sollte aber besser strip() verwenden, weil hier wirklich alle "Leerzeichen" entfernt werden.	Returns a string whose value is this string, with all leading and trailing white space removed. Example: (" A B\n ").strip() is "A B".

"GIRAFFEN"
01234567